



Plantation Tracking System

Requirements & Scope Document

Android App Development using GenAI • April 2026

Project Type	Domain / Category	Timeline
Student Android Project	Environmental Tech	2 Weeks

1. Introduction

Every year, thousands of plantation drives are organised across India by schools, NGOs, government bodies, and community groups. Volunteers gather on a chosen day, plant saplings or seed balls in open land, take photographs, and leave. While the intention behind these drives is noble, the critical follow-up phase — ensuring those saplings actually survive and grow — almost never happens.

The core problem is the absence of tracking. Once a plantation event ends, there is no system in place to:

- Record where exactly each plant was placed
- Know what species was planted and when
- Monitor whether the plant is alive, growing, or dead
- Measure the actual survival rate of any plantation drive

Without this data, the true ecological impact of plantation drives remains unknown. A 1,000-tree plantation event may have a 20% survival rate, meaning 800 plants died unnoticed — but the community believes they planted 1,000 trees. This false sense of success prevents improvement and wastes resources.

Namma-Hasiru (meaning "Our Green" in Kannada) is a simple, offline-first Android application designed to solve this exact problem. It gives every green volunteer a personal tool to log the plants they add to the earth, check on them over time, and contribute to a growing picture of what truly thrives in their region.

2. Objectives

The Namma-Hasiru app is built around the following core objectives:

- **Plant Logging:** Enable Geo-Tagged Planting Records
- **Status Monitoring:** Allow volunteers to check in on plant health at defined intervals (e.g., 90 days)
- **Survival Analytics:** Provide an aggregated survival rate score at the individual and community level

- **Growth Documentation:** Facilitate photo-based visual progress tracking so users can see plant growth over time
- **Species Intelligence:** Generate insights on which plant species perform best in a given locality
- **User Engagement:** Motivate consistent follow-up behaviour through reminders and progress badges
- **Data for Impact:** Build a foundational citizen-science dataset for ecological research

3. Scope of the Project

3.1 What the App Will Include (In-Scope)

- A form to add a new plant entry with name, species, photo, date, and optional location tag
- A local database (Room DB) to store all plant records persistently on the device
- A dashboard listing all plants with their current status (Alive / Dead / Unknown)
- The ability to update a plant's status and attach a new photo at any point
- A basic survival score calculated as a percentage of "Alive" plants out of total logged
- A 90-day reminder system using Android Work Manager to prompt the user for a status check
- A species reference guide with suggested trees based on locally recorded success data
- A clean, green-themed UI that is intuitive for first-time users

3.2 What the App Will NOT Include (Out-of-Scope)

- No cloud backend or real-time server sync — all data is stored locally on the device
- No user authentication or login system — single-user mode only for this version
- No live map or satellite imagery — location is stored as coordinates, not displayed on a map
- No social or community sharing features — volunteers cannot see each other's records
- No AI-powered image analysis to assess plant health from photos
- No multi-language support — the app will function in English only

4. Functional Requirements

The table below lists all functional requirements with their priority level.

Feature	Description	Priority
Add New Plant	Capture plant name, species, date planted, and an optional photo. Assign a unique plant ID automatically.	Must Have
View Plant List	Display all logged plants in a scrollable list with their name, date, and current status badge.	Must Have
Plant Detail View	Tap any plant to see its full history: all photos, status changes, notes, and planting date.	Must Have

Update Plant Status	Mark a plant as Alive, Dead, or Unknown. Attach an updated photo and add a short note.	Must Have
Local Data Storage	All data is saved using Room Database. Records persist across app restarts and device reboots.	Must Have
Survival Score	Auto-calculated percentage of Alive plants. Displayed prominently on the home dashboard.	Must Have
90-Day Reminder	WorkManager schedules a background notification 90 days after a plant is added, prompting a status update.	Must Have
Species Guide	A static reference list of tree species recommended for the local region with care tips.	Should Have
Growth Photo Timeline	View all photos attached to a plant in chronological order to visually track growth.	Should Have
Filter & Sort	Filter the plant list by status (Alive/Dead/Unknown) or sort by planting date or name.	Nice to Have
GPS Geo-Tag (Optional)	If location permission is granted, record GPS coordinates at the time of planting using FusedLocationProviderClient.	Nice to Have

4.1 Detailed Feature Descriptions

Add New Plant: This is the primary action in the app. A floating action button (FAB) opens a form where the user fills in the common name, scientific name (optional), date (auto-filled to today), and captures a photo from the camera or gallery. On submission, a new record is created in the local Room Database and the 90-day reminder is scheduled.

Track & Update Status: Each plant has a status field that begins as "Unknown." At any point, the user can open the plant detail screen and tap "Update Status" to change it to Alive, Dead, or Unknown, add an optional note, and optionally take a new photo. Every update is timestamped and stored in the history log for that plant.

Survival Score: The home dashboard shows a large, circular progress indicator showing the community survival rate. The formula is: $\text{Survival Rate} = (\text{Number of plants marked Alive} / \text{Total plants with a status update}) \times 100\%$. Plants still at "Unknown" are excluded from the calculation to keep the number accurate.

90-Day Reminder: When a plant is added, a Work Manager periodic work request is scheduled to fire after 90 days. The notification reads: "Time to check on your [Plant Name]! Is it still growing?" Tapping the notification opens the plant detail screen directly.

5. Non-Functional Requirements

Requirement	Specification
Performance	The app should launch in under 2 seconds on a mid-range Android device. The plant list should scroll at a consistent 60 fps without lag, even with 200+ records.
Usability	A first-time user should be able to log their first plant within 60 seconds without reading any instructions. All buttons must be clearly labelled. Font sizes must be accessible.
Reliability	No data loss may occur due to the app crashing or the device being restarted. Room DB transactions must be atomic and rollback-safe.
Offline Functionality	The app must work entirely without an internet connection. 100% of all features — including photo capture, status updates, and reminders — must function offline.
Compatibility	The app must support Android 8.0 (API Level 26) and above to cover the majority of devices in rural India.
Storage	Photos are stored internally on the device. The app must handle storage gracefully and warn the user if available space is critically low.

6. System Overview

Namma-Hasiru is a single-device, offline-first Android application. There is no server, cloud, or internet dependency. All data flows within the device itself.

User Interaction Layer: The user interacts with the app through Activities and Fragments built using standard Android Views or Jetpack Compose. The main screens are: Home Dashboard, Add Plant, Plant List, Plant Detail, Update Status, and Species Guide.

Business Logic Layer: A ViewModel (following MVVM architecture) holds the UI state and calls Repository methods. The Repository is the single source of truth, fetching data from the Room Database and providing it to the ViewModel via LiveData or StateFlow.

Data Storage Layer: Room Database (built on top of SQLite) stores all plant records. Two primary tables are used: Plants (master record per plant) and StatusHistory (one record per update, linked by plant ID). Photos are saved to the app's internal file storage and their file paths are stored in the database.

Background Tasks: Work Manager manages the 90-day reminder. When a plant is added, a OneTimeWorkRequest is enqueued with a delay of 90 days. Even if the device is restarted, Work Manager re-schedules the work automatically, making the reminder highly reliable.

7. Project Constraints

Constraint	Impact & Mitigation
2-Week Timeline	Only core features (plant logging, status tracking, survival score, reminder) will be built. Advanced features like maps and analytics are deferred to a future phase.
No Backend / Server	All data is device-local. There is no community-wide data aggregation in this phase. Multi-user collaboration is out of scope.
No Real GPS (Phase 1)	GPS geo-tagging is optional. In Phase 1, location capture may be simulated or skipped. If the user denies location permission, the app continues to function normally.
Simulated Test Data	Since the 90-day reminder cannot be tested in real time, test builds will use a 30-second delay to verify Work Manager functionality. The production build will use the full 90-day delay.
Student Development Team	The development team consists of students with limited professional Android experience. The architecture will be kept simple and well-documented to allow for smooth collaboration.

8. Future Enhancements

The following features are planned for future phases after the core application has been built and tested:

- **Phase 2: GPS Map View:** Interactive map view showing all geotagged plants as pins, color-coded by status
- **Phase 2: Enhanced Notifications:** Smart push notifications and in-app reminders at 30, 60, and 90 days post-planting
- **Phase 3: Cloud Sync:** Community data pooling via a lightweight cloud backend (Firebase) to aggregate survival data across volunteers
- **Phase 3: Analytics Dashboard:** Dashboard charts showing survival trends by species, season, and locality over time
- **Phase 4: Smart Suggestions:** AI-powered species recommendation engine that suggests the most suitable trees for a given GPS zone based on historical survival data
- **Phase 4: Visual Health AI:** Camera-based plant health check that uses image recognition to flag unhealthy saplings automatically
- **Phase 4: Gamification:** Gamification layer with volunteer badges, streaks, and leaderboards to drive long-term engagement
- **Phase 5: Localisation:** Multi-language support for Kannada, Hindi, Tamil, and Telugu to make the app accessible to rural volunteers

9. Conclusion

Namma-Hasiru is more than a student project — it is a practical response to one of the most overlooked problems in environmental activism: the gap between planting a tree and growing one. Millions of saplings are planted each year with genuine care and good intent, yet most fail silently because no one is watching.

This application gives every green volunteer a simple, reliable tool to close that loop. By logging plants, tracking their survival, and documenting their growth through photos, users transform a one-time act of planting into an ongoing commitment to stewardship. Over time, the data generated by the app will paint a clear, evidence-based picture of what works — which species thrive, which locations succeed, and which practices make a real difference.

Built within a realistic two-week student timeline using standard Android technologies — Room Database, WorkManager, and FusedLocationProviderClient — the app is designed to be simple to build, easy to extend, and genuinely useful from day one. It requires no internet, no login, and no special knowledge to use, which means it can reach the volunteers who need it most.

Every tree logged in Namma-Hasiru is a promise kept — not just to the earth, but to the community that planted it. This document defines the foundation for turning that promise into a measurable, scalable, and lasting reality.